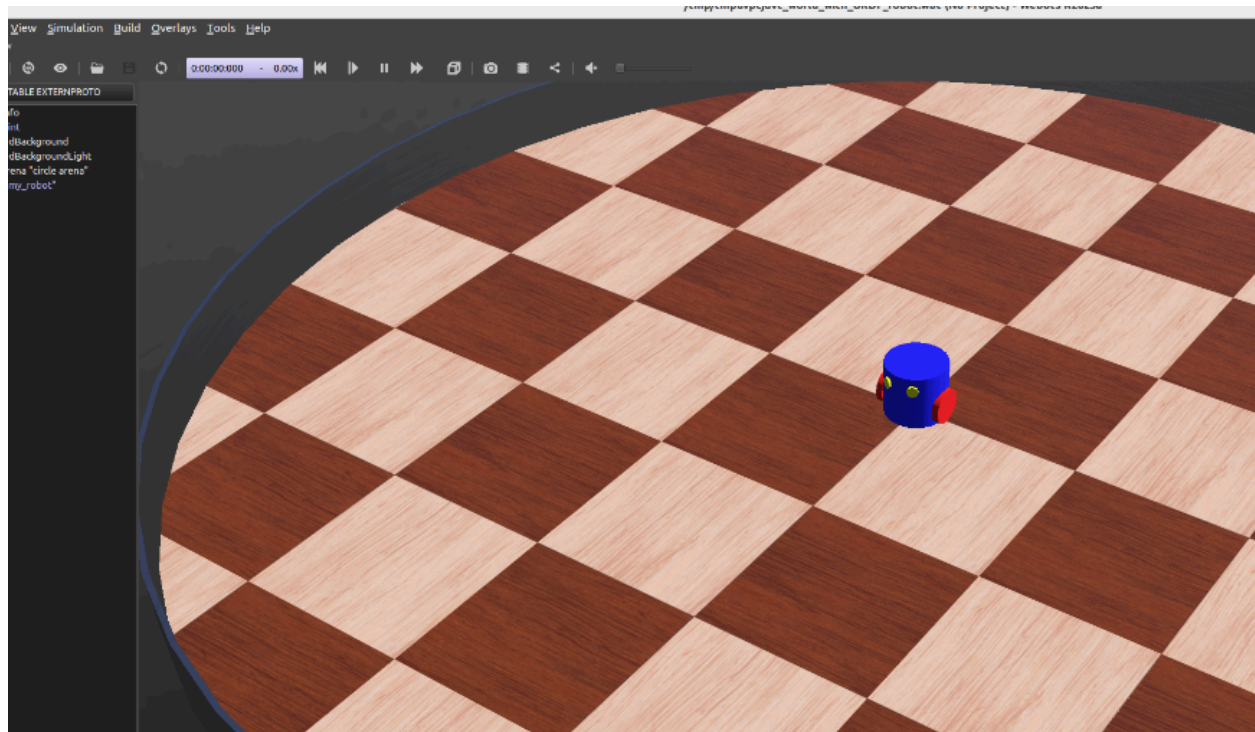
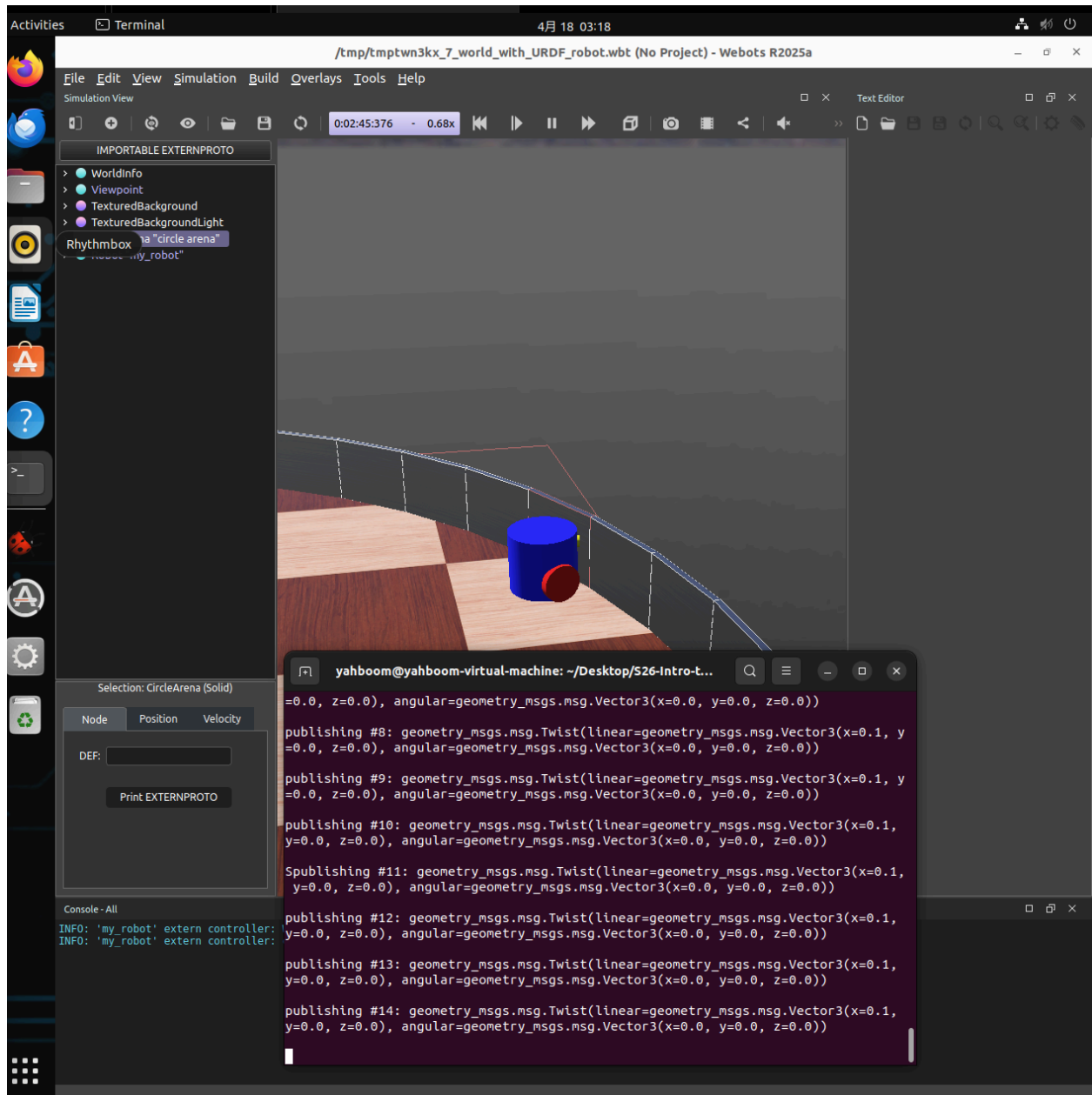


Prelab Step 9

Required Question: Based on this file, what can you determine about the function of this node?
(e.g. what topics are utilized; what motion is possible)

This node acts as the Webots driver for the robot. It subscribes to the `cmd_vel` topic with a `geometry_msgs/Twist` message, then uses the linear and angular velocity commands to calculate the left and right wheel speeds. From that, we can tell the robot uses differential drive, so it can move forward, backward, turn, and rotate in place. It does not support sideways motion, and in the code shown its main ROS role is subscribing to `cmd_vel`.

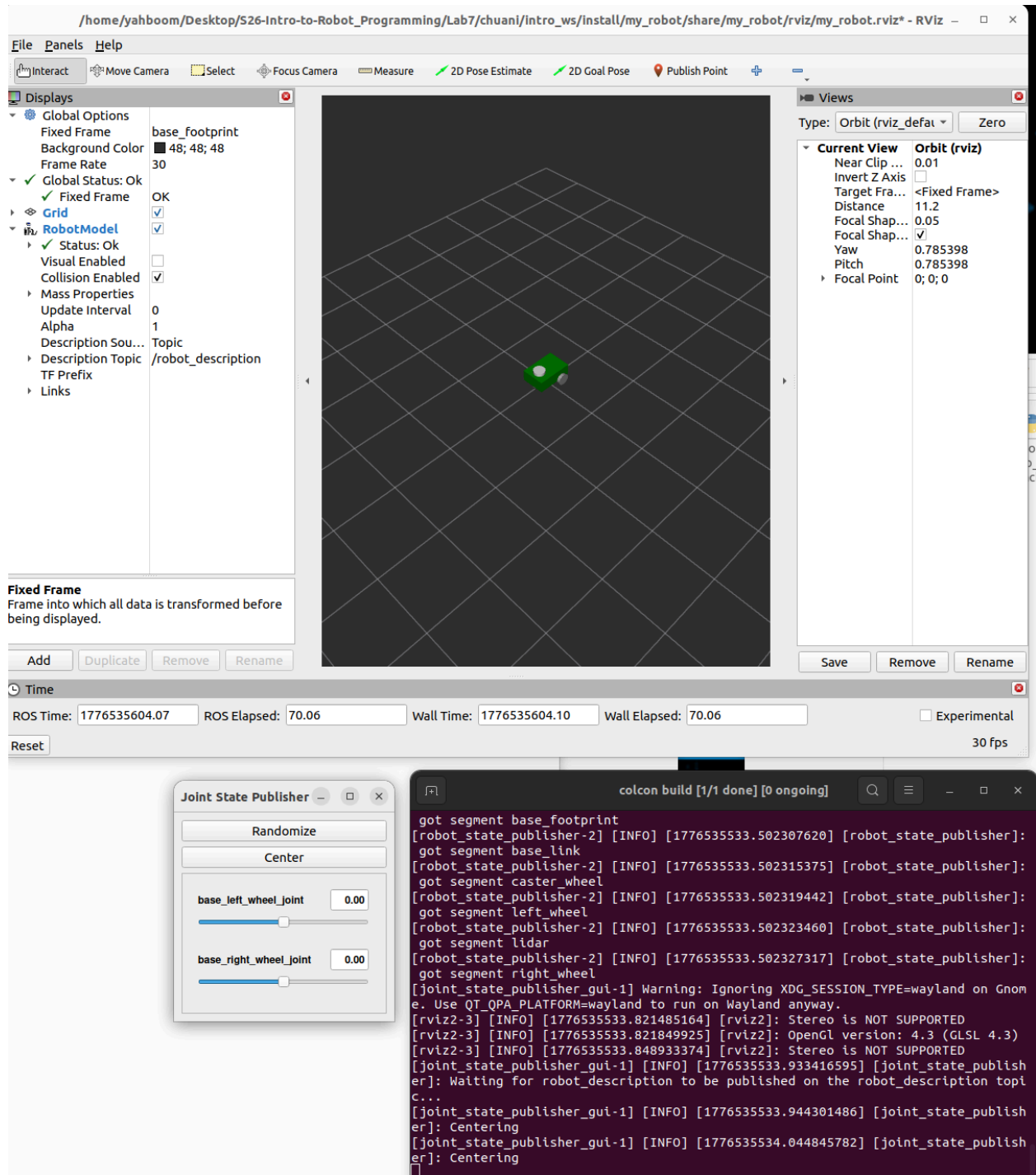




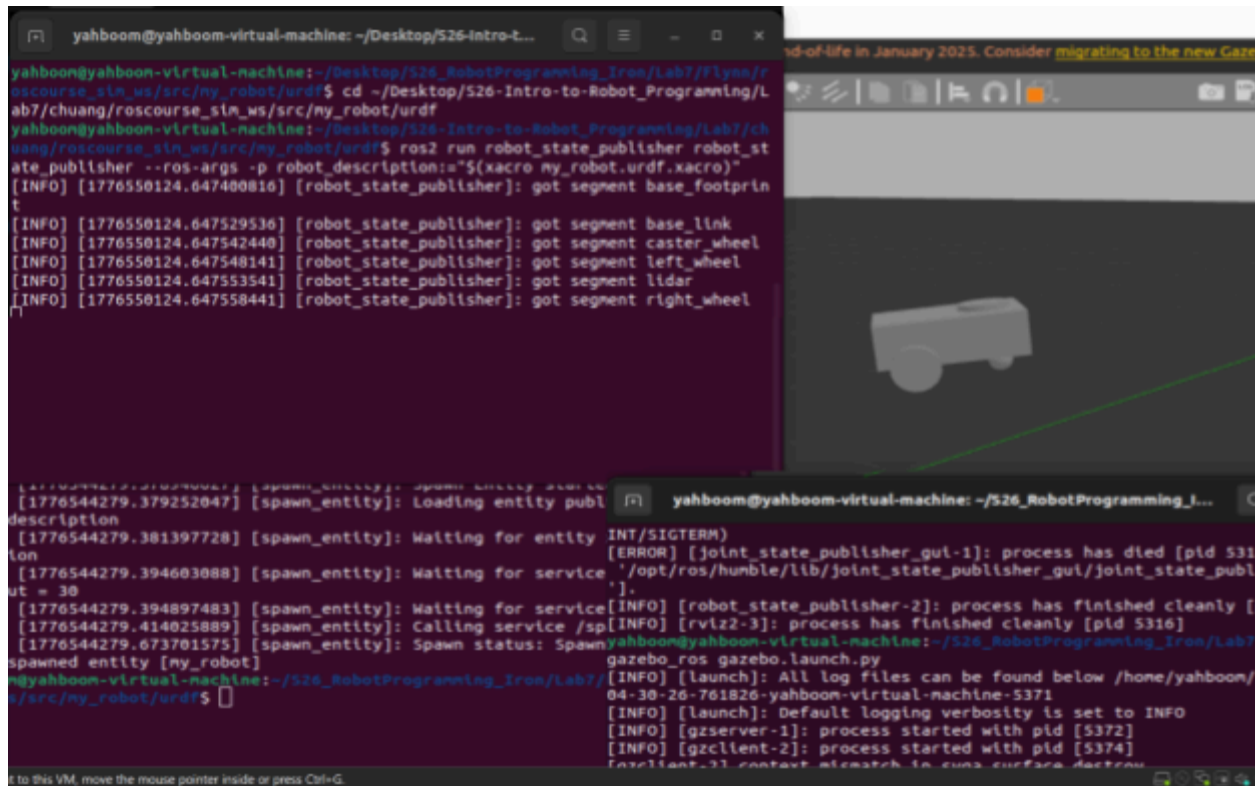
Webot moving forward, as seen from the terminal, bumping into the wall.

Lab 7

- [] Task 1 Step 10 with Rviz and command in terminal visible



- [] Terminals and Gazebo window from Task 3 Step 2



- [] Completed launch file from Task 3 Step 7

```

import os
fromament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import Command, LaunchConfiguration
from launch.conditions import IfCondition, UnlessCondition
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from launch_ros.parameter_descriptions import ParameterValue
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import IncludeLaunchDescription

def generate_launch_description():

    my_robot_path = get_package_share_directory('my_robot')
    urdf = os.path.join(my_robot_path, 'my_robot.urdf.xacro')
    rviz_config = os.path.join(my_robot_path, 'my_robot.rviz')

    use_sim_time = LaunchConfiguration('use_sim_time', default='false')

    sim_time_arg = DeclareLaunchArgument(
        name='use_sim_time',
        default_value='true', choices=['true', 'false'],
        description='Use simulation (Gazebo) clock if true')

    rviz_arg = DeclareLaunchArgument(name='rvizconfig', default_value=str(rviz_config),
        description='Absolute path to rviz config file')

    robot_desc=ParameterValue(Command(['xacro ',urdf]),value_type=str)

    # Depending on gui parameter, either launch joint_state_publisher or joint_state_publisher_gui

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',

```

```

    name='robot_state_publisher',
    output='screen',
    parameters=[{'use_sim_time': use_sim_time, 'robot_description': robot_desc}],
    arguments=[urdf])

rviz_node = Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    output='screen',
    arguments=['-d', LaunchConfiguration('rvizconfig')])

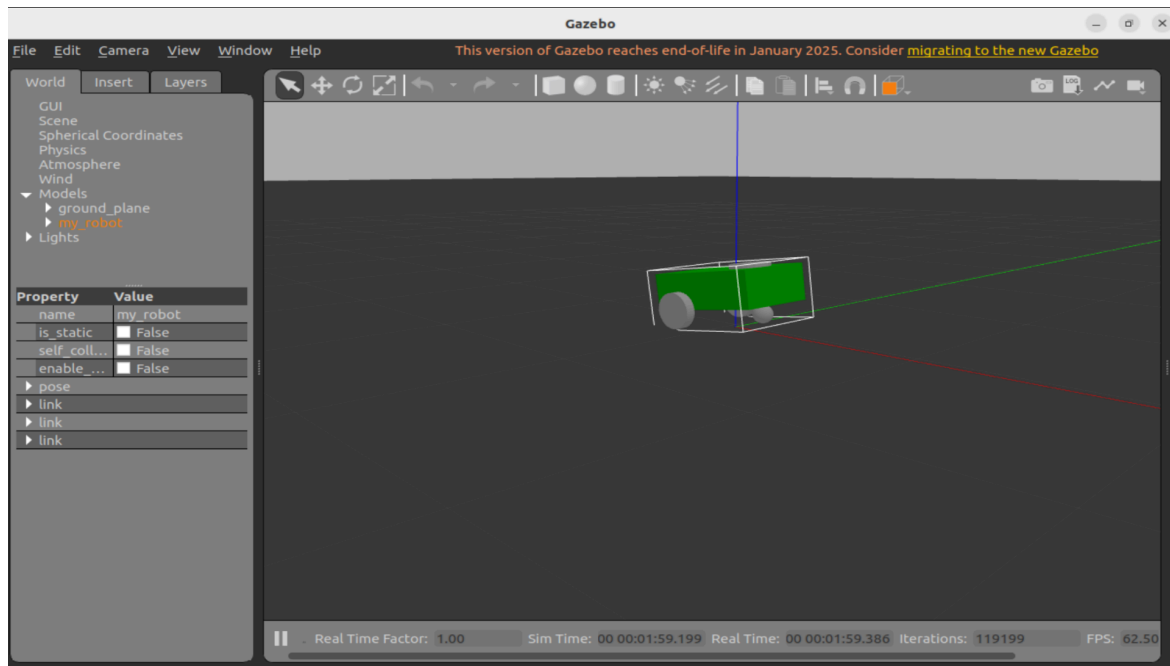
gazebo_launch =
IncludeLaunchDescription(PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('gazebo_ros'),'launch/gazebo.launch.py')))

gazebo_robot = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=['-topic', 'robot_description', '-entity', 'my_robot'])

return LaunchDescription([
    sim_time_arg,
    rviz_arg,
    robot_state_publisher_node,
    rviz_node,
    gazebo_launch,
    gazebo_robot
])

```

- [] Gazebo window after running the launch file for Task 4 Step 8



- [] Completed urdf.xacro file

```

<?xml version="1.0"?>
<robot name="my_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <material name="green">
    <color rgba="0 0.6 0 1"/>
  </material>

  <material name="white">
    <color rgba="1 1 1 1"/>
  </material>

```

```

<material name="gray">
  <color rgba=".7 .7 .7 1"/>
</material>
<xacro:property name="base_length" value="0.6" />
<xacro:property name="base_width" value="0.4" />
<xacro:property name="base_height" value="0.2" />
<xacro:property name="wheel_length" value="0.05" />
<xacro:property name="wheel_radius" value="0.1" />

<xacro:macro name="box_inertia" params="mass lx ly lz xyz rpy">
  <inertial>
    <origin xyz="{xyz}" rpy="{rpy}" />
    <mass value="{mass}" />
    <inertia ixx="{(mass / 12.0) * (ly * ly + lz * lz)}" ixy="0" ixz="0"
      iyy="{(mass / 12.0) * (lx * lx + lz * lz)}" iyz="0"
      izz="{(mass / 12.0) * (lx * lx + ly * ly)}" />
  </inertial>
</xacro:macro>

<xacro:macro name="cylinder_inertia" params="mass r h xyz rpy">
  <inertial>
    <origin xyz="{xyz}" rpy="{rpy}" />
    <mass value="{mass}" />
    <inertia ixx="{(mass / 12.0) * (3 * r * r + h * h)}" ixy="0" ixz="0"
      iyy="{(mass / 12.0) * (3 * r * r + h * h)}" iyz="0"
      izz="{(mass / 2.0) * (r * r)}" />
  </inertial>
</xacro:macro>

<xacro:macro name="sphere_inertia" params="mass r xyz rpy">
  <inertial>
    <origin xyz="{xyz}" rpy="{rpy}" />
    <mass value="{mass}" />
    <inertia ixx="{(2.0 / 5.0) * mass * (r * r)}" ixy="0" ixz="0"
      iyy="{(2.0 / 5.0) * mass * (r * r)}" iyz="0"
      izz="{(2.0 / 5.0) * mass * (r * r)}" />
  </inertial>
</xacro:macro>

<link name="base_link">
  <visual>
    <geometry>
      <box size="{base_length} {base_width} {base_height}" />
    </geometry>
    <origin xyz="0 0 .1" rpy="0 0 0"/>
    <material name="green"/>
  </visual>
  <collision>
    <geometry>
      <box size="{base_length} {base_width} {base_height}" />
    </geometry>
    <origin xyz="0 0 {base_height / 2.0}" rpy="0 0 0" />
  </collision>
  <xacro:box_inertia mass="5.0" lx="{base_length}" ly="{base_width}" lz="{base_height}" xyz="0 0 {base_height / 2.0}"
    rpy="0 0 0" />
</link>

<link name="lidar">
  <visual>
    <geometry>
      <cylinder radius="{base_height/2.0}" length="0.05" />

```

```

        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="white"/>
    </visual>
    <collision>
    <geometry>
        <cylinder radius="0.1" length="0.05" />
    </geometry>
    <origin xyz="0 0 0" rpy="0 0 0" />
</collision>
</link>

<link name="left_wheel">
    <visual>
        <geometry>
            <cylinder radius="${wheel_radius}" length="${wheel_length}" />
        </geometry>
        <origin xyz="0 0 0.025" rpy="0 0 0" />
        <material name="gray"/>
    </visual>
    <collision>
        <geometry>
            <cylinder radius="${wheel_radius}" length="${wheel_length}" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
    </collision>
    <xacro:cylinder_inertia mass="1.0" h="0.05" r="0.1" xyz="0 0 0" rpy="0 0 0"/>

</link>

<link name="right_wheel">
    <visual>
        <geometry>
            <cylinder radius="${wheel_radius}" length="${wheel_length}" />
        </geometry>
        <origin xyz="0 0 -0.025" rpy="0 0 0" />
        <material name="gray"/>
    </visual>
    <collision>
        <geometry>
            <cylinder radius="${wheel_radius}" length="${wheel_length}" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
    </collision>
    <xacro:cylinder_inertia mass="1.0" h="0.05" r="0.1" xyz="0 0 0" rpy="0 0 0"/>

</link>

<link name="caster_wheel">
    <visual>
        <geometry>
            <sphere radius="${wheel_length}" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="gray" />
    </visual>
    <collision>
        <geometry>
            <sphere radius="${wheel_radius}" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
    </collision>

```

```

        <xacro:sphere_inertia mass="2.0" r="{wheel_radius}" xyz="0 0 0" rpy="0 0 0"/>
    </link>

    <link name="base_footprint"/>

    <joint name="base_joint" type="fixed">
        <parent link="base_footprint" />
        <child link="base_link" />
        <origin xyz="0 0 0.1" rpy="0 0 0" />
    </joint>

    <joint name="base_lidar_joint" type="fixed">
        <parent link="base_link"/>
        <child link="lidar"/>
        <origin xyz="0.15 0 .2" rpy="0 0 0"/>
    </joint>

    <joint name="base_left_wheel_joint" type="continuous">
        <parent link="base_link"/>
        <child link="left_wheel"/>
        <origin xyz="-0.15 0.2 0" rpy="{-pi / 2.0} 0 0"/>
        <axis xyz="0 0 1"/>
    </joint>

    <joint name="base_right_wheel_joint" type="continuous">
        <parent link="base_link"/>
        <child link="right_wheel"/>
        <origin xyz="-0.15 -0.2 0" rpy="{-pi / 2.0} 0 0"/>
        <axis xyz="0 0 1"/>
    </joint>

    <joint name="base_caster_wheel_joint" type="fixed">
        <parent link="base_link" />
        <child link="caster_wheel"/>
        <origin xyz="0.2 0 -0.05" rpy="0 0 0" />
    </joint>
    <gazebo reference="base_link">
    <material>Gazebo/Green</material>
    </gazebo>

    <gazebo>
    <plugin name="diff_drive_controller" filename="libgazebo_ros_diff_drive.so">
        <!-- Update rate in Hz -->
        <update_rate>50</update_rate>

        <!-- wheels -->
        <left_joint>base_left_wheel_joint</left_joint>
        <right_joint>base_right_wheel_joint</right_joint>

        <!-- kinematics -->
        <wheel_separation>0.4</wheel_separation>
        <wheel_diameter>0.2</wheel_diameter>

        <!-- output -->
        <publish_odom>true</publish_odom>
        <publish_odom_tf>true</publish_odom_tf>
        <publish_wheel_tf>true</publish_wheel_tf>

        <odometry_topic>odom</odometry_topic>
        <odometry_frame>odom</odometry_frame>
        <robot_base_frame>base_footprint</robot_base_frame>
    </gazebo>

```



```
<!-- input -->  
<command_topic>cmd_vel</command_topic>  
  
</plugin>  
</gazebo>  
</robot>
```